

P10 Cash drawer and Day Close

Model: Opus, high effort.

Branch: none. Commit directly to `main`.

Depends on: P09.

1. What to build, in one sentence

Build the cash drawer, the one function that computes a day's figures, the first balance checks, Day Close with a blind cash count and a lock that keeps a closed day closed, and a golden day fixture that replays all of 26 September through the real API and proves every Day Close number.

2. Module

The cash drawer and Day Close parts of M16.

It also wires the day lock into M3, M10, M16 and M17 endpoints built earlier.

3. The spec is already written

`docs/API-CONTRACT.md` section "M16 Settlement and Day Close", and `docs/DB-SCHEMA.md` sections 24 and 25.

Build exactly what they say. If the spec and this prompt disagree, the spec wins, and you tell me.

If the spec is silent on something you need, stop and ask.

4. Step 0. Save this prompt and check the repo

1. Save this entire prompt, exactly as given, to `docs/prompts/P10-cash-drawer-day-close.md`.
2. `git pull` . `git status` should show only that file.
3. `docs/prompts/README.md` must show P09 as Done. If not, stop.
4. Run `npm test` and record the count. It should match P09's "after" count. If anything fails, stop and tell me.

5. Files to read first

1. The cash drawer and Day Close parts of the M16 spec.
 2. `docs/REPORT-SPEC.md` R2 and R7.
 3. `docs/RECONCILIATION-RULES.md` , all of it.
 4. `docs/TEST-DATA.md` , all of it. Section 4 is the acceptance test for this prompt.
 5. `docs/GLOSSARY.md` sections 4 and 10.
 6. `server/utils/time.js` with the test clock, `server/utils/money.js` , `server/utils/tax.js` .
 7. Every service that writes to bills, payments, orders, accounts, cash or payouts. You will add the lock to each.
 8. The receipt layout code, for the Day Close print.
 9. `seedDemo.js` , for how this repo drives the real API from a script.
-

6. Part A. Cash drawer

Build, exactly as specified:

the `CashMovement` model, registered,
the three endpoints with their permissions,
the one live opening float per business date,
the `CASH_PAID_OUT` audit line,
and voiding with a reason.

`businessDate` always comes from `nowUtc()` and
`businessDateFor` . No request may set it.

7. Part B. Day figures

Build `server/services/dayFiguresService.js` with
`computeDayFigures(req, businessDate, { session })` .
It returns R2 sections A to H from `docs/REPORT-SPEC.md` ,
using the definitions in `docs/GLOSSARY.md` , from frozen
fields only:

Section	Reads
A. Sales	Non-voided bills with this <code>businessDate</code>
B. Where the bill total went	Those bills' payments by frozen method, grouped by frozen kind, their <code>chargedToAccountInPaise</code> by account, and any still unpaid
C. Collections	<code>accountentries</code> of type <code>COLLECTION</code> with this <code>businessDate</code>

Section	Reads
D. Cash drawer	<code>cashmovements</code> for this date, cash payments whose own <code>businessDate</code> is this date, and cash collections
E. Order types	Bills grouped by <code>orderType</code> , delivery split by frozen <code>platform.code</code>
F. GST by rate	Bills' <code>taxBreakdown</code> , with <code>PLATFORM_COLLECTS</code> bills in their own 0% row
G. Controls	Discounts from bills, No Charge from orders with <code>noCharge.businessDate</code> on this date, cancelled lines and whole orders by the business date of their cancel time, voided bills
H. Invoices	Per <code>invoiceSeries</code> : first, last, issued, voided, gaps

Rules:

1. Use MongoDB aggregation where it helps, but every sum is in whole paise and every grouping uses frozen fields. Never read the menu, categories, users or payment methods to build a figure.
2. Averages are a sum divided by a sum, rounded half away from zero to the paisa, the same rule as `money.js` .
3. "Cash" means payments and collections whose frozen `methodKind` is `IN_HAND` and whose method code is

CASH .

4. The return value is plain data. The same object is stored as the Day Close snapshot, and will be returned by the R2 report in P15.

8. Part C. The first balance checks

Build `server/services/reconciliationService.js` with checks for one business date:

C1, C3, C4, C6, C8 and C9, exactly as

`docs/RECONCILIATION-RULES.md` and the M16 spec define them.

Each check returns `{ id, severity, passed, message, expected, actual, difference, refs }`, where `refs` lists the bill numbers or record ids behind a failure.

Messages use the exact wording in `docs/RECONCILIATION-RULES.md`, with the values filled in.

C6 for one day checks, per invoice series: no gaps and no repeats between the lowest and highest sequence issued that day, and the lowest is exactly one more than the highest sequence of any earlier bill in that series, when there is one.

P14 adds C2, C5, C7, C10, C11, C12, and the range versions. Leave the file organised so adding them is obvious.

9. Part D. Day Close

Build, exactly as specified:

the `DayClosure` model, registered,
the five endpoints,

the blockers, reported together in one 422 `DAY_NOT_READY` ,
the note required for a cash difference,
the snapshot and checks stored at close,
the history,
the `DAY_CLOSED` and `DAY_REOPENED` audit lines,
the `dayClose.showCashDifferenceToManager` setting,
default false,
and the blind count: a MANAGER's responses leave out
expected cash and the difference unless that setting is on.

The Day Close print, `GET /day-close/:businessDate/print?width=32` , is laid out on the
server with the same code style as the receipt: sections A,
B, D and G, the check results, and "Closed by {name} at
{time}". At width 32 a manager's print follows the same
blind rule as the screen.

10. Part E. The lock

Build `assertDayOpen(req, businessDate, { session })`
in a small service of its own, reading `dayclosures` .
It throws 409 `DAY_CLOSED` with the spec's message when
that date is `CLOSED` . A `REOPENED` date is open.

Wire it into every write listed in the M16 spec's lock section.
At minimum:

Write	Date checked
Create a bill	The business date the new bill would get

Write	Date checked
Discount, void, take a payment, correct a payment, charge to account	The bill's businessDate
Take a payment, correct a payment	Also the payment's own business date, today's
No Charge	Today's business date
Cash movement, and voiding one	Its business date
Account collection, adjustment	Today's business date
Record or void a payout	Today's business date

Remove the comment P08 left at the payment correction, now that the check is there.

Inside a transaction, call it inside the transaction.

11. Part F. The golden day fixture

Build `server/tests/helpers/goldenDay.js`.

It builds the whole golden day from `docs/TEST-DATA.md` through the real API, with the test clock set to each event's time, the same way `seedDemo.js` drives the API, so every rule runs exactly as in production.

In order:

1. A restaurant with business day start 300, invoice mode PREFIX , prefix CFA/C/ , starting number 22442, inventory and attendance off.
2. Stations Live Kitchen and Beverages, the 13 categories with their stations, and the 22 menu items at the exact prices in section 1.
3. Staff: Owner, Manager, Counter as CASHIER, and the four captains as WAITER.
4. Payment methods ZOMATO_GOLD , DINEOUT , EAZYDINER , ZOMATO linked to ZOMATO , and SWIGGY linked to SWIGGY , all PLATFORM , plus the built-in Cash, Card and UPI.
5. Accounts "E-210 Office" and "W-330 Office", opening at 0.
6. Opening float 200000 at 11:00 AM.
7. Bills B01 to B16, each opened by its captain at its opened time, fired, billed and paid at the times in section 2, with the exact discounts, reason codes and payments. B11 is voided at 8:09 PM with WRONG_TABLE , then the same order is billed again as B12. B13's order has its two cancelled items. B14 is paid at 12:02 AM on 27 September. B09 and B10 are charged to their accounts.
8. N01, the No Charge on Table 29.
9. Paid out 35000 at 9:30 PM by the Manager, reason "Milk from the dairy".

It returns the ids and tokens a test needs, and exports the expected figures from docs/TEST-DATA.md section 4 as plain constants, so every later report test can reuse both.

Keep it fast. Build it once per test file, not once per test.

12. The client

1. **Cash drawer screen**, for OWNER, MANAGER and CASHIER: today's float, paid in and paid out, each with who and when, buttons to add each one, and void for a manager. A cashier does not see the paid out button.
2. **Day Close screen**, for OWNER and MANAGER:
 1. Pick the business date, defaulting to the one that most needs closing.
 2. If anything blocks the close, list each blocker as a plain sentence with a link to the order or bill.
 3. One field: "Cash counted in the drawer". No expected figure shown to a manager.
 4. A note field, which becomes required when the server says the count differs.
 5. After closing: the day's figures, the checks, and "Print". The owner also sees expected cash and the difference.
 6. For the owner on a closed day: "Reopen", with a required reason.
3. **Dashboard**: when yesterday's business date is not closed, a plain warning with a link to Day Close.

Use the printing code from P05 for the print. Follow `docs/DESIGN-SYSTEM.md` .

13. Tests

The acceptance test, in a new file

`server/tests/goldenDay.test.js`

1. Build the golden day. Close 26 September as the Manager with counted cash 340000 and a note.
2. The stored snapshot matches every number in `docs/TEST-DATA.md` section 4 for R2 sections A, B, D, E, F, G and H, to the paisa: 15 bills, 27 covers, item total 931022, discount 42390, net sales 888632, CGST 19131, SGST 19126, GST 38257, round-off 11, bill total 926900, average bill 59242, average per cover 26709, every payment method total, On Hold 4700 and 50400, expected cash 340400, difference -400, and the rest.
3. Checks C1, C3, C4, C6 and C8 pass. C9 raises its warning for the -400 difference.
4. The Manager's response leaves out expected cash and the difference. The Owner's includes them.

Cash drawer

5. A second opening float on the same date is refused. After voiding the first, it is accepted.
6. A cashier may record a float and a paid in, and is refused a paid out.

Day Close

7. An open order on the date blocks the close. An unpaid bill blocks it. Both appear together in one response.
8. A cash difference without a note is refused.
9. Closing twice is refused. Reopening needs the owner and a reason. After reopening and closing again, the history has three entries.

Lock

10. After closing 26 September, each write in the Part E table that targets 26 September is refused with 409

`DAY_CLOSED` , one test per row.

11. After reopening, the same writes succeed.

12. Writes for 27 September are never affected by closing the 26th.

Checks

13. For each of C1, C3, C4, C6 and C8, a copy of the golden day with exactly one thing broken, from `docs/TEST-DATA.md` section 6, fails exactly that check. Break the stored documents directly in the test database, after the fixture has run.

Run the full suite at the end. Every test that passed before must still pass.

14. Non-negotiable rules that apply

"Reports only add up values frozen onto records when the event happened."

"Store all money as whole paise integers."

"Which business day a moment belongs to is decided only by `businessDateFor`."

"Never hard delete." Cash movements are voided, closes are reopened, nothing is removed.

"Check permissions on the server for every endpoint." The blind count is enforced in the response, on the server.

15. Checks and golden day

This prompt is where the golden day first runs end to end.
Test 2 is the most important test in the project so far.
If any number in test 2 differs from `docs/TEST-DATA.md`, do not change the expected number to make it pass. Find out which side is wrong and tell me. The expected numbers were produced by the repo's own tax code.

16. Docs to update

1. `docs/RECONCILIATION-RULES.md` section 1: C1, C3, C4, C6, C8 and C9 are built for one business date, in P10.
 2. `docs/PROJECT-STATE.md` :
 1. The date line.
 2. "Current stage": "Next: P11, Caffeza setup and menu import."
 3. Module status: M16 becomes DONE when every test above passes, with the note "Day Close proven against the golden day in `tests/goldenDay.test.js`."
 4. Every detail where the build differed from the spec, as a decision row, and the spec updated to match.
 5. "What changed recently": a P10 entry at the top, and the oldest moved to the archive.
 3. `docs/prompts/README.md` : mark P10 as Done.
-

17. Out of scope

Report screens. R2 as a report arrives in P15, using

`computeDayFigures` .

Checks C2, C5, C7, C10, C11 and C12. That is P14.

Shifts, or more than one close per day.

Counting cash by denomination.

18. Done when

1. `npm test` passes, with before and after counts recorded, including the golden day acceptance test.
2. `npm run lint` and `npm run build` pass.
3. By hand: record a float, take a cash bill, record a paid out, close the day as a manager without seeing the expected cash, then as the owner see the difference and print the close.
4. Every doc in section 16 is updated.
5. Commits on `main` , one line each, for example:
 - `add cash drawer`
 - `add day figures and first checks`
 - `add day close with blind count`
 - `lock closed days`
 - `add golden day fixture and acceptance test`
 - `update docs for p10`
6. Push `main` .
7. Print a short summary: commits, files changed, test counts before and after, the golden day snapshot's

section A exactly as stored, and every place the build differed from the spec.